MTD215

Timed Online Assignment – July Semester 2021

Application of C++ in Multimedia

Tuesday, 23 November 2021 **1:00 pm – 3:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **EIGHT (8)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, your Full Name, and SUSS PI Number. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAXYZ_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

Question 1

(a) Implement a C++ program that demonstrates the appropriate syntax for constructing data structures such as arrays and pointers. These data structures form part of the data members and constructors in a C++ class.
Declare the data members of `Cart` class as follows:

(i) An `integer` representing the ID of the cart.

(ii) A `string` representing the owner of the cart

(iii) An `integer` representing the quantity of cart item.

(iv) A dynamic location large enough to store all the items in the cart. The location is reference by a pointer `string* items`.

(4 marks)

(b) Implement an application using the C++ language in an object-oriented style. Constructors and destructors are used to initialise and remove objects in an object-oriented manner.

You are asked to write the following based on the above `Cart` class:

(i) A default constructor.
(Assuming that there are at least 2 items in the cart, you may use any valid default values for the data members)

(5 marks)

(ii) A parameterised constructor.

(5 marks)

(iii) A destructor.

(2 marks)

(c) Create a friend function `displayCart` in the `Cart` class. The friend function will display the details of the cart data members. The example output is shown in Figure Q1(c).

```
In Friend function
Card Id: 123 Card owner name: Mary Tan Number of items: 3 in cart
Items are:
Pen
Pencil
Eraser
```

Figure Q1(c): Example output of displayCart

(6 marks)

(d) Write a `main()` function to demonstrate how the default and parameterised constructors in part (b) and friend function in part (c) are being used.

(3 marks)

Question 2

Identify suitable coding modules in a simple program that implement polymorphism. In this question you are asked to apply operator overloading which is a form of polymorphism.

An instance of `Cylinder` can be represented by `radius` and `height`. Both `radius` and `height` are integers.

Implement a C++ class to model `Cylinder` as specified below:

(a) Declare the data members `radius` and `height`.

(2 marks)

(b) Write a parameterised constructor.

(3 marks)

(c) Create a `volume` function that calculates the volume of the cylinder. The formula of the area of the cylinder is $volume=\pi r^2 h$.

(3 marks)

(d) Overload the >> operator to allow user to enter the radius and height of the cylinder. The example of getting user inputs is shown in Figure Q2(e).

```
Cylinder Details:
Enter radius: 5
Enter height: 8
```

Figure Q2(e): Example of getting the user inputs

(5 marks)

(e) Overload the << operator so that the sample statement

```
cout << "c1 = " << c1;
```

will produce the result

```
Radius of Cylinder: 4 and height of Cylinder: 6
Volume of Cylinder: 301.593
```

(5 marks)

(f) Overload the > operator to check if an instance of `Cylinder` is larger (in terms of volume) than another instance of `Cylinder`.

(4 marks)

(g) Write the `main()` function to get the number of cylinder(s) from user input and construct `Cylinder` instances based on the number of cylinder that the user input. Demonstrate the use of the >>, << and > operators. An example of output is shown in Figure Q2(g).

```
Enter the number of Cylinder: 2
Cylinder Details:
Enter radius: 4
Enter height: 6
Cylinder Details:
Enter radius: 5
Enter height: 8

Radius of Cylinder: 4 and height of Cylinder: 6
Volume of Cylinder: 301.593

Radius of Cylinder: 5 and height of Cylinder: 8
Volume of Cylinder: 628.319

The largest volume of Cylinder is: 628.319
Program ended with exit code: 0
```

Figure Q2(g): Example of output of main function

(8 marks)

Question 3

Discuss the concept of arrays and inheritance by answering the following questions.

(a) Assume the following definition and initialisation in the main function:

```
string months[12] = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov",
"Dec"};

int weather2020[12] = {25,24,36,23,18,20,18,24,18,32,35,36};
```

The `months` array stores the short form of the months and `weather2020` array stores the temperature of each month.

Create a main function and write C++ statements to carry out the following tasks and explain how you derive your solution.

- Output the average temperature for all months.
- Output the lowest temperature month(s).
- Output the highest temperature month(s).

The example output is shown in Figure Q3(a).

```
Average weather temperature for 2020: 25
The lowest temperature month(s): May Jul Sep
The highest temperature month(s):Mar Dec
```

Figure Q3(a) – Output from main function

(9 marks)

(b) Study the following implemented `Bank` class.

```
class Bank {
private:
   int bankCode;
   string bankName;
   int branchCode;
public:
   Bank(int, string, int);
   void display();
};
Bank::Bank(int bc, string bn, int branch) {
   bankCode = bc;
   bankName = bn;
   branchCode = branch;
}
void Bank::display() {
   cout << "Bank Name: " << bankName << " Brunch Code: " << branchCode << endl;
}
```

Implemented a `BankAccount` class which inherits `Bank` class.

(i) Declare the data members of `BankAccount` class as follows:

- An `integer` representing the account number.
- A `string` representing the account user name.
- An `integer` representing the account type.

(3 marks)

(ii) Write a parameterised constructor for `BankAccount` class. The constructor will also call it's base class `(Bank)` constructor to initiatise the base class data members.

(6 marks)

(iii) Write a `display` function for `BankAccount` class to display the values of the data members from the `Bank` class and the `BankAccount` class. The example output is shown in Figure Q3(b).

```
Bank Name: ABC Bank Branch Code: 1
Account Name: Ann Lim Account Number: 12345
```

Figure Q3(b) – Output from display function

(5 marks)

(iv) Write a `main` function to demonstrate inheritance concept of `Bank` class and `BankAccount` class.

(2 marks)

Question 4

Appraise the use of SDL for geometry rendering. SDL has primitive functions for drawing point, lines and rectangles. Consider the following code section in the `main function` which is responsible for producing the output in Figure Q4(a).

```
const int SCREEN_WIDTH = 600;
const int SCREEN_HEIGHT = 360;

const int offset = 140;
const int limit = 4;

SDL_Window* gWindow = NULL;
SDL_Renderer* gRenderer = NULL;

int main(int argc, const char *argv[]) {

   SDL_Init(SDL_INIT_VIDEO);
   gWindow = SDL_CreateWindow("Student Name and PI", SDL_WINDOWPOS_UNDEFINED,
SDL_WINDOWPOS_UNDEFINED, SCREEN_WIDTH, SCREEN_HEIGHT, SDL_WINDOW_SHOWN);

   gRenderer = SDL_CreateRenderer(gWindow, -1, SDL_RENDERER_ACCELERATED);
   SDL_SetRenderDrawColor(gRenderer, 0xFF, 0xFF, 0xFF, 0xFF);

   SDL_Event event;
   bool quit = false;

   while(!quit) {
      while (SDL_PollEvent(&event)) {
         if (event.type == SDL_QUIT) {
            quit = true;
         }

         SDL_SetRenderDrawColor(gRenderer, 0xFF, 0xFF, 0xFF, 0xFF);
         SDL_RenderClear(gRenderer);

         int offsetX = 0;

         SDL_SetRenderDrawColor(gRenderer, 51, 51, 255, 225);
         SDL_RenderDrawLine(gRenderer, 50 + offsetX, 10, 10 + offsetX, 90);
         SDL_RenderDrawLine(gRenderer, 50 + offsetX, 10, 90 + offsetX, 90);
         SDL_RenderDrawLine(gRenderer, 10 + offsetX, 90, 90 + offsetX, 90);


         SDL_SetRenderDrawColor(gRenderer, 255, 51, 51, 225);
         SDL_RenderDrawLine(gRenderer, 30 + offsetX, 110, 70 + offsetX, 110);
         SDL_RenderDrawLine(gRenderer, 30 + offsetX, 110, 10 + offsetX, 190);
         SDL_RenderDrawLine(gRenderer, 10 + offsetX, 190, 90 + offsetX, 190);
         SDL_RenderDrawLine(gRenderer, 90 + offsetX, 190, 70 + offsetX, 110);

         SDL_RenderPresent(gRenderer);
      }
   }

   SDL_DestroyRenderer(gRenderer);
   SDL_DestroyWindow(gWindow);
   gWindow = NULL;
   gRenderer = NULL;
   SDL_Quit();
}
```

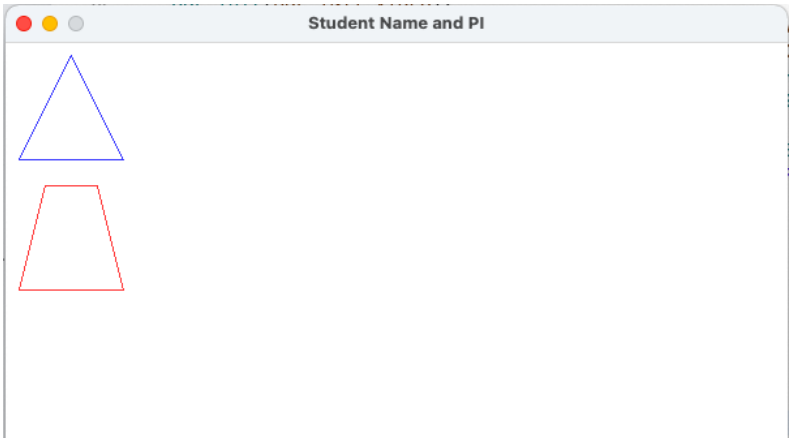

Figure Q4(a)

Re-write the `main` function to produce the following output in Figure Q4(b):

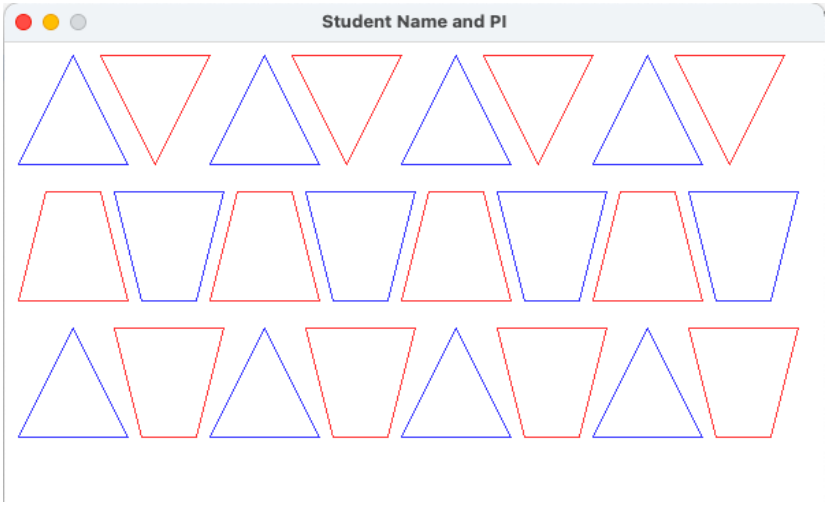

Figure Q4(b)

(i) Eight Triangles as shown in the output of Figure Q4(b).

(6 marks)

(ii) Eight Irregular Rectangles as shown in the output of Figure Q4(b).

(6 marks)

(iii) A mixture of 4 Triangles and 4 Irregular Rectangle as shown in the output of Figure Q4(b).

(8 marks)

Replace the title of the window "`Student Name and PI`" with your name and id. The position of each primitive should be as close as possible to the above figure.

---- END OF PAPER ----